

Unsupervised Learning

Definition:

Unsupervised Learning is a type of machine learning in which the model learns patterns and relationships from data without using labeled outputs.

The algorithm analyzes the data and automatically discovers hidden structures, similarities, or groups. Clustering is one of the most common tasks in unsupervised learning.

K-Means Clustering

Definition:

K-Means is a clustering algorithm that divides data into **K clusters** based on similarity.

The algorithm assigns data points to the nearest cluster center (centroid) and repeatedly updates the centroids until stable clusters are formed. It is simple, fast, and effective for well-separated clusters.

The number of clusters (**K**) must be specified before training.

Hierarchical Clustering

Definition:

Hierarchical Clustering is a clustering technique that builds a hierarchy of clusters by repeatedly merging or splitting groups of data points.

It starts by treating each data point as an individual cluster and gradually combines the closest clusters. The results are displayed using a **dendrogram**, which helps determine the appropriate number of clusters.

Provides a visual representation of cluster relationships.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Definition:

DBSCAN is a density-based clustering algorithm that groups together closely packed data points and identifies isolated points as outliers.

The algorithm forms clusters based on data density rather than distance to a centroid. It can discover clusters of different shapes and automatically detect noise points.

Does not require the number of clusters to be specified in advance and can identify outliers.

Principal Component Analysis (PCA)

The dataset contains multiple dimensions, making visualization difficult.

Principal Component Analysis (PCA) was applied to reduce the dimensionality of the dataset while preserving most of the important information.

The data was transformed into two principal components:

- PC1
- PC2

These components were then used to visualize clustering results in a two-dimensional space.

```
In [53]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score

from scipy.cluster.hierarchy import dendrogram, fcluster, linkage

from sklearn.decomposition import PCA

import warnings
warnings.filterwarnings("ignore")

plt.style.use('default')
```

```
In [54]: df = pd.read_csv("Country-data.csv")

df.head()
```

```
Out[54]:
```

	country	child_mort	exports	health	imports	income	inflation	life_
0	Afghanistan	90.2	10.0	7.58	44.9	1610	9.44	
1	Albania	16.6	28.0	6.55	48.6	9930	4.49	
2	Algeria	27.3	38.4	4.17	31.4	12900	16.10	
3	Angola	119.0	62.3	2.85	42.9	5900	22.40	
4	Antigua and Barbuda	10.3	45.5	6.03	58.9	19100	1.44	

```
In [55]: print("Shape:", df.shape)

df.info()

df.describe()
```

```

Shape: (167, 10)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 167 entries, 0 to 166
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   country         167 non-null    object
1   child_mort       167 non-null    float64
2   exports         167 non-null    float64
3   health          167 non-null    float64
4   imports         167 non-null    float64
5   income          167 non-null    int64
6   inflation        167 non-null    float64
7   life_expec      167 non-null    float64
8   total_fer       167 non-null    float64
9   gdpp            167 non-null    int64
dtypes: float64(7), int64(2), object(1)
memory usage: 13.2+ KB

```

```

Out[55]:

```

	child_mort	exports	health	imports	income	
count	167.000000	167.000000	167.000000	167.000000	167.000000	16
mean	38.270060	41.108976	6.815689	46.890215	17144.688623	
std	40.328931	27.412010	2.746837	24.209589	19278.067698	1
min	2.600000	0.109000	1.810000	0.065900	609.000000	-
25%	8.250000	23.800000	4.920000	30.200000	3355.000000	
50%	19.300000	35.000000	6.320000	43.300000	9960.000000	!
75%	62.100000	51.350000	8.600000	58.750000	22800.000000	10
max	208.000000	200.000000	17.900000	174.000000	125000.000000	104

```

In [56]: countries = df["country"]
X = df.drop('country', axis = 1)

```

```

In [57]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(
    X_scaled,
    columns = X.columns
)

X_scaled.head()

```

```
Out[57]:
```

	child_mort	exports	health	imports	income	inflation	life_exp
0	1.291532	-1.138280	0.279088	-0.082455	-0.808245	0.157336	-1.67
1	-0.538949	-0.479658	-0.097016	0.070837	-0.375369	-0.312347	0.64
2	-0.272833	-0.099122	-0.966073	-0.641762	-0.220844	0.789274	0.67
3	2.007808	0.775381	-1.448071	-0.165315	-0.585043	1.387054	-1.17
4	-0.695634	0.160668	-0.286894	0.497568	0.101732	-0.601749	0.70

KMeans CLustering

```
In [58]: inertia = []

for k in range(1,11):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

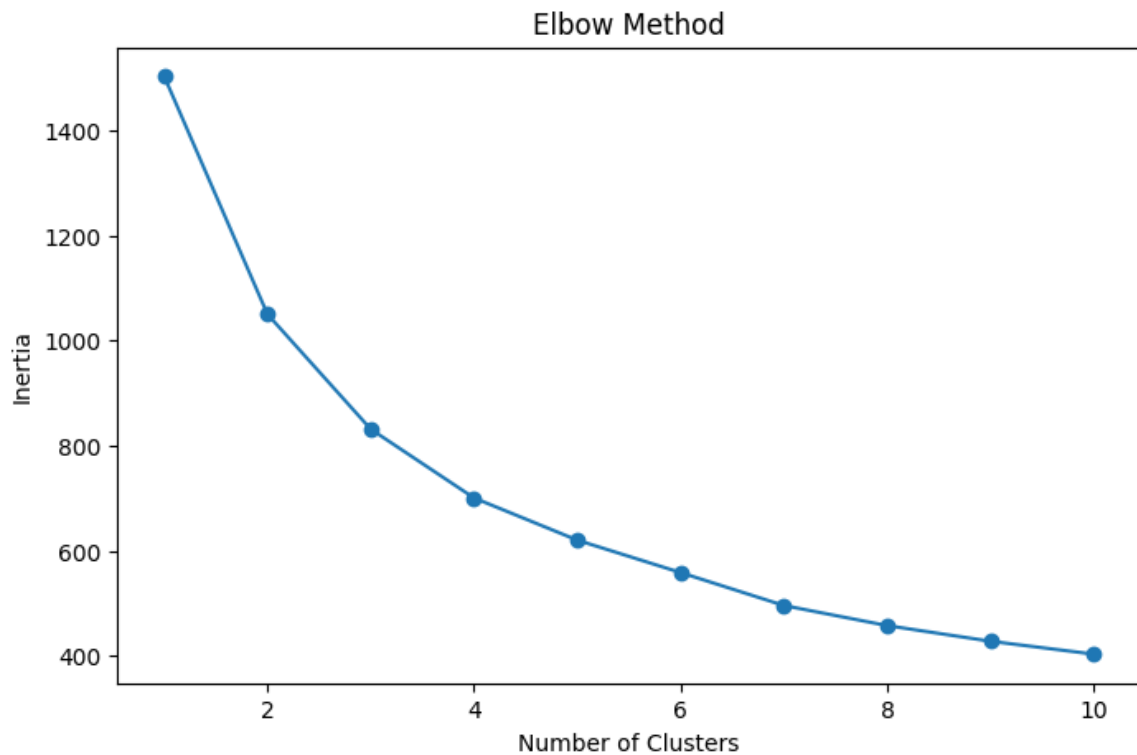
    model.fit(X_scaled)

    inertia.append(model.inertia_)
```

```
In [59]: plt.figure(figsize=(8,5))

plt.plot(range(1,11), inertia, marker='o')

plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```



```
In [60]: kmeans = KMeans(  
          n_clusters=3,  
          random_state=42,  
          n_init=10  
        )  
  
kmeans_labels = kmeans.fit_predict(X_scaled)  
  
df['KMeans_Cluster'] = kmeans_labels
```

```
In [61]: score = silhouette_score(  
          X_scaled,  
          kmeans_labels  
        )  
  
print("Silhouette Score:", round(score,3))
```

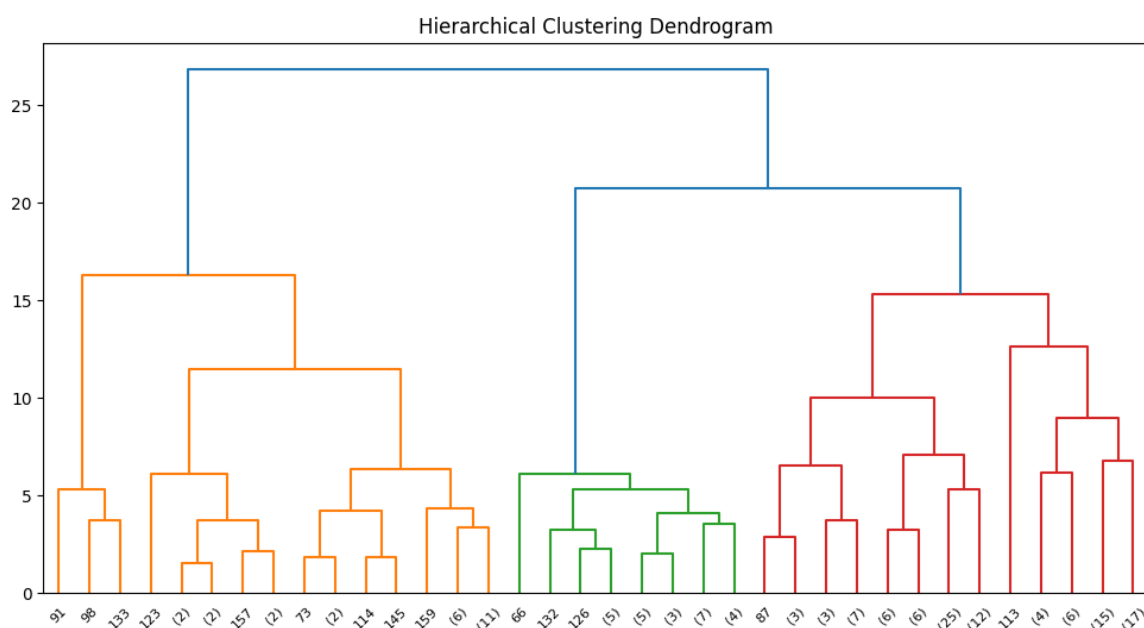
Silhouette Score: 0.283

Hierarchical Clustering

```
In [62]: linkage_matrix = linkage(  
          X_scaled,  
          method='ward'  
        )  
  
plt.figure(figsize=(12,6))  
  
dendrogram(  
    linkage_matrix,  
    truncate_mode='level',  
    p=5
```

```
)

plt.title("Hierarchical Clustering Dendrogram")
plt.show()
```



```
In [63]: hierarchical_labels = fcluster(
        linkage_matrix,
        t=3,
        criterion='maxclust'
    )

df['Hierarchical_Cluster'] = hierarchical_labels
```

```
In [64]: score = silhouette_score(
        X_scaled,
        hierarchical_labels
    )

print("Silhouette Score:", round(score,3))
```

Silhouette Score: 0.246

DBSCAN Clustering

```
In [65]: dbscan = DBSCAN(
        eps=1.2,
        min_samples=5
    )

dbscan_labels = dbscan.fit_predict(X_scaled)

df['DBSCAN_Cluster'] = dbscan_labels
```

```
In [66]: n_clusters = len(set(dbscan_labels))

if -1 in dbscan_labels:
```

```
n_clusters -= 1

print("Clusters Found:", n_clusters)

print("Noise Points:",
      list(dbscan_labels).count(-1))
```

Clusters Found: 3

Noise Points: 53

```
In [67]: if len(set(dbscan_labels)) > 1:
          score = silhouette_score(
              X_scaled,
              dbscan_labels
          )

          print("Silhouette Score:",
                round(score,3))
      else:
          print("Silhouette Score cannot be computed.")
```

Silhouette Score: 0.148

```
In [68]: pca = PCA(n_components=2)

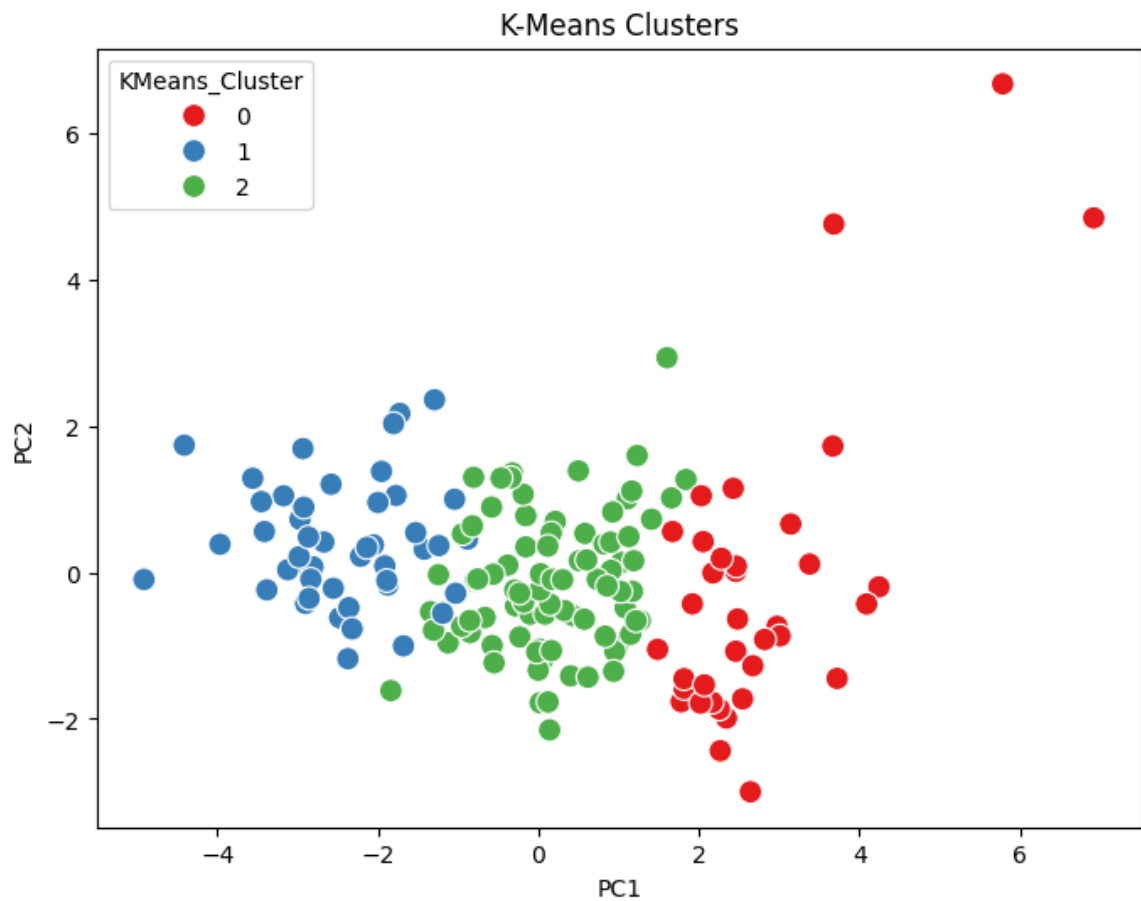
X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(
    X_pca,
    columns=['PC1', 'PC2']
)
```

```
In [69]: plt.figure(figsize=(8,6))

sns.scatterplot(
    x='PC1',
    y='PC2',
    hue=df['KMeans_Cluster'],
    palette='Set1',
    data=pca_df,
    s=100
)

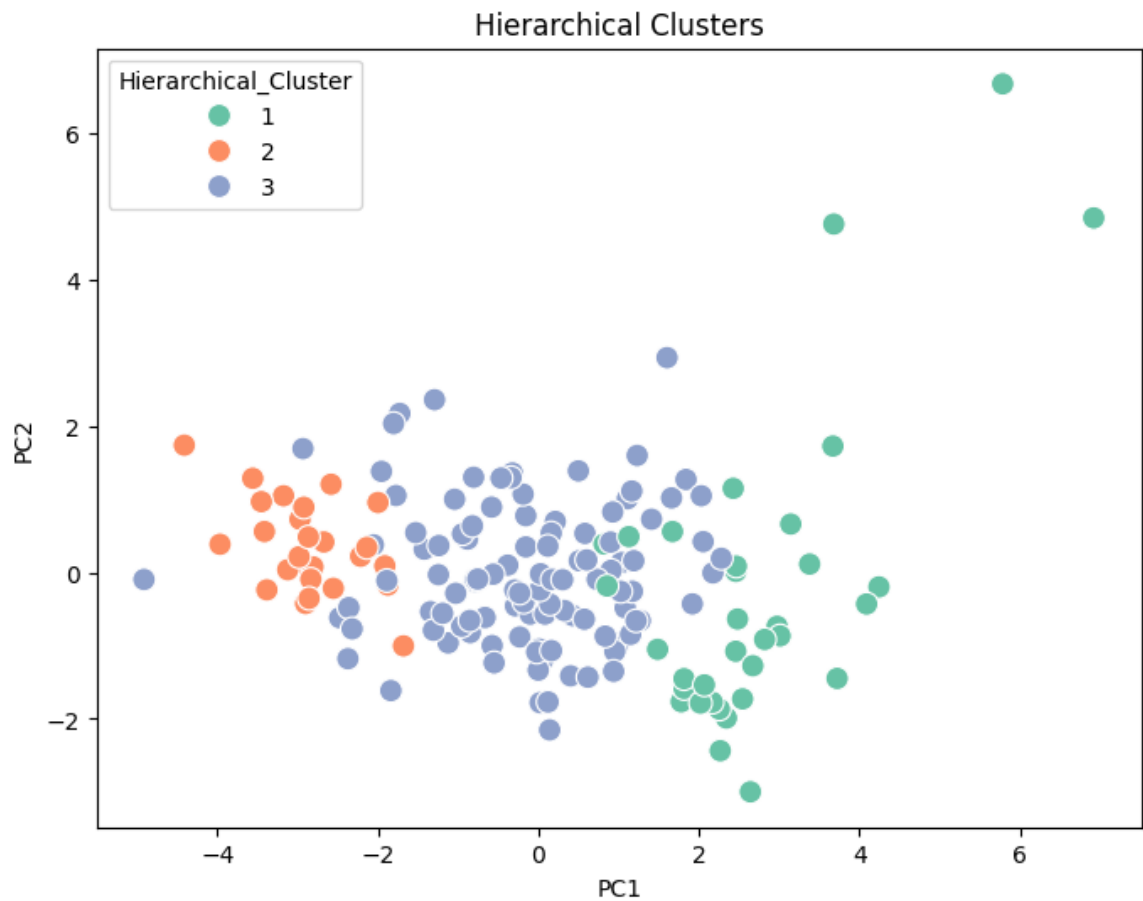
plt.title("K-Means Clusters")
plt.show()
```

```
In [70]: plt.figure(figsize=(8,6))

sns.scatterplot(
    x='PC1',
    y='PC2',
    hue=df['Hierarchical_Cluster'],
    palette='Set2',
    data=pca_df,
    s=100
)

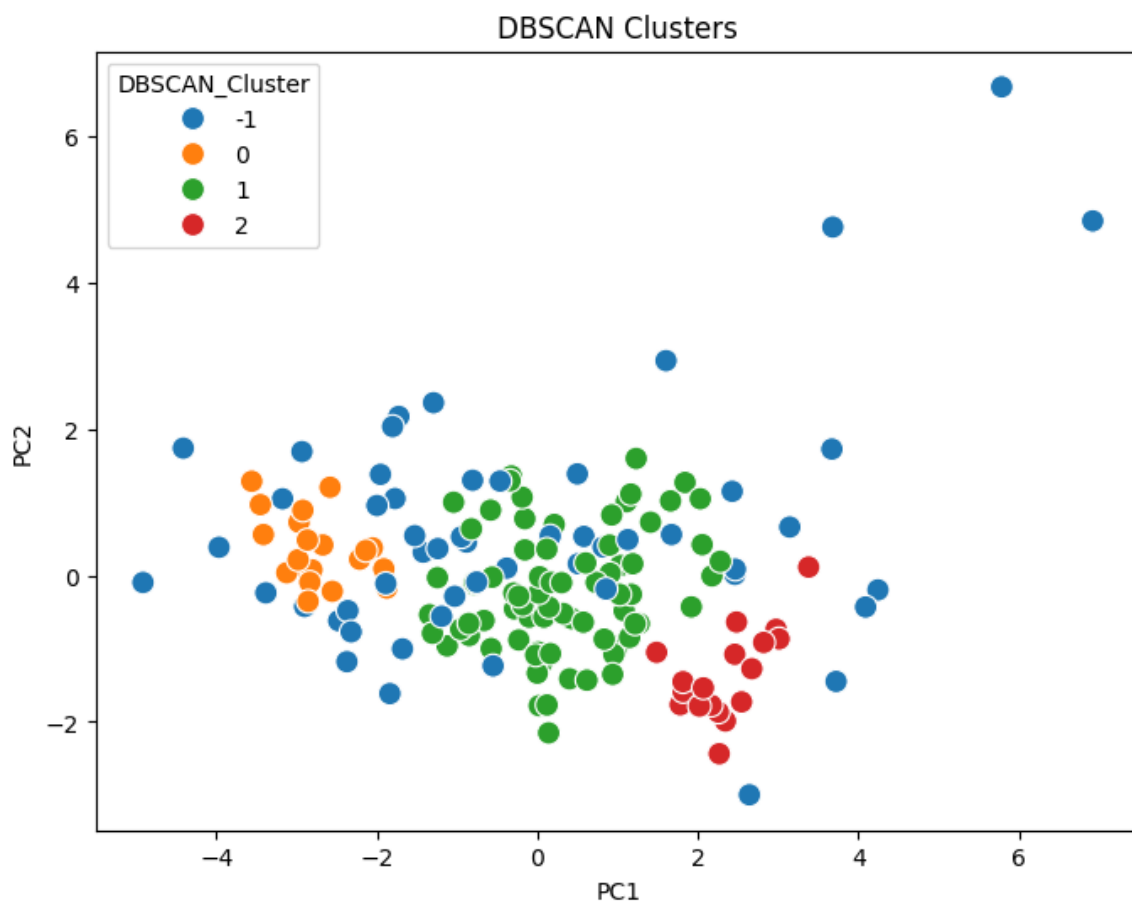
plt.title("Hierarchical Clusters")
plt.show()
```



```
In [71]: plt.figure(figsize=(8,6))

sns.scatterplot(
    x='PC1',
    y='PC2',
    hue=df['DBSCAN_Cluster'],
    palette='tab10',
    data=pca_df,
    s=100
)

plt.title("DBSCAN Clusters")
plt.show()
```



```
In [72]: cluster_summary = df.groupby(
          'KMeans_Cluster'
        ).mean(numeric_only=True)

cluster_summary
```

```
Out[72]:
```

	child_mort	exports	health	imports	income
KMeans_Cluster					
0	5.000000	58.738889	8.807778	51.491667	45672.222222
1	92.961702	29.151277	6.388511	42.323404	3942.404255
2	21.927381	40.243917	6.200952	47.473404	12305.595238

```
In [73]: cluster_summary = df.groupby(
          'Hierarchical_Cluster'
        ).mean(numeric_only=True)

cluster_summary
```

Out[73]:

	child_mort	exports	health	imports	inc
Hierarchical_Cluster					
1	5.961765	58.508824	8.501176	48.902941	47588.23
2	105.070370	23.589630	6.507037	39.662963	1589.74
3	31.617925	39.990368	6.353679	48.085527	11341.88

```
In [74]: cluster_summary = df.groupby(
        'DBSCAN_Cluster'
    ).mean(numeric_only=True)

cluster_summary
```

Out[74]:

	child_mort	exports	health	imports	income
DBSCAN_Cluster					
-1	51.816981	50.783377	6.763208	54.657847	22391.15094
0	96.719048	24.776190	5.679048	38.809524	1830.57142
1	20.514667	40.161067	6.366267	46.638667	12878.53333
2	4.172222	35.627778	10.168889	34.494444	37338.88888

```
In [75]: for cluster in sorted(df['KMeans_Cluster'].unique()):

        print("\nCluster", cluster)

        print(
            df[df['KMeans_Cluster']==cluster]
              ['country']
              .head(10)
              .tolist()
        )
```

Cluster 0

['Australia', 'Austria', 'Bahrain', 'Belgium', 'Brunei', 'Canada', 'Cyprus', 'Czech Republic', 'Denmark', 'Finland']

Cluster 1

['Afghanistan', 'Angola', 'Benin', 'Botswana', 'Burkina Faso', 'Burundi', 'Cameroon', 'Central African Republic', 'Chad', 'Comoros']

Cluster 2

['Albania', 'Algeria', 'Antigua and Barbuda', 'Argentina', 'Armenia', 'Azerbaijan', 'Bahamas', 'Bangladesh', 'Barbados', 'Belarus']

```
In [76]: results = pd.DataFrame({
        "Country": countries,
        "KMeans": df["KMeans_Cluster"],
        "Hierarchical": df["Hierarchical_Cluster"],
        "DBSCAN": df["DBSCAN_Cluster"]
    })
```

```
results.head()
```

Out[76]:

	Country	KMeans	Hierarchical	DBSCAN
0	Afghanistan	1	2	0
1	Albania	2	3	1
2	Algeria	2	3	1
3	Angola	1	3	-1
4	Antigua and Barbuda	2	3	1